

# Interpretable Reinforcement Learning for Load Balancing using Kolmogorov-Arnold Networks

Kamal Singh<sup>1</sup>, Sami Marouani<sup>1</sup>, Ahmad Al Sheikh<sup>2</sup>, Pham Tran Anh Quang<sup>3</sup>, Amaury Habrard<sup>1,4,5</sup>

<sup>1</sup>Université Jean Monnet Saint-Étienne, CNRS, Inst. d'Optique Graduate School, Lab. Hubert Curien, F-42023 Saint-Étienne, France

<sup>2</sup>QoS Design, Toulouse, France

<sup>3</sup>Huawei Technologies Ltd., Paris Research Center, France

<sup>4</sup>Institut Universitaire de France (IUF) <sup>5</sup>Inria

Email: <sup>1</sup>{firstname.surname}@univ-st-etienne.fr, <sup>2</sup>alsheikh@qosdesign.com, <sup>3</sup>phamt.quang@huawei.com

**Abstract**—Reinforcement learning (RL) has been increasingly applied to network control problems, such as load balancing. However, existing RL approaches often suffer from lack of interpretability and difficulty in extracting controller equations. In this paper, we propose the use of Kolmogorov-Arnold Networks (KAN) for interpretable RL in network control. We employ a PPO agent with a 1-layer actor KAN model and an MLP Critic network to learn load balancing policies that maximise throughput utility, minimize loss as well as delay. Our approach allows us to extract controller equations from the learned neural networks, providing insights into the decision-making process. We evaluate our approach using different reward functions demonstrating its capability in improving network performance while providing interpretable policies.

## I. INTRODUCTION

Network providers face increasing complexity and must support diverse traffic, necessitating agile, autonomous networks to manage tasks like routing and QoS while controlling costs. This requires new algorithms for decision-making and control. Existing methods relying on optimization or control theory require full system knowledge, which is challenging. Meanwhile, ML approaches, though effective, often lack interpretability—a critical barrier for deployment in sensitive environments where failures can be costly.

We focus on load balancing over Software-Defined Wide Area Network (SD-WAN) where traffic flows between origin-destination pairs can be distributed across multiple logical paths. We propose an interpretable Deep Reinforcement Learning approach integrating a Kolmogorov Arnold Network (KAN) [1] within a Proximal Policy Optimization framework to obtain load balancing policies which enhance throughput and reduce packet loss. Our method extracts symbolic policies, offering both performance and interpretability essential for network operators. Our source code is available online<sup>1</sup>.

### A. Related work

Traffic Engineering (TE) optimizes network resource utilization while meeting performance requirements like throughput and reliability. Traditional mechanisms, such as MPLS-TE [2], rely heavily on signaling protocols like RSVP-TE [3], which limits flexibility in dynamic environments. Segment Routing over IPv6 (SRv6) [4] enhances modern TE by leveraging the IPv6 stack for flexible, stateless packet forwarding. It uses

the Segment Routing Header (SRH) to guide packets along predetermined paths without requiring complex control protocols like RSVP or LDP, reducing core overhead. Oblivious TE methods like shortest-path routing often lead to congestion. Semi-oblivious approaches such as COPE [5] improve link utilization but face computational overhead and sensitivity to traffic changes.

Deep Reinforcement Learning (DRL) has shown promise in optimizing network QoS metrics, with works including [6] for delay minimization. Most DRL-based TE methods lack interpretability and overlook safety during training and inference, risking link capacity violations. Constrained approaches like RCPO [7] offer soft guarantees through Lagrangian methods, while Control Barrier Functions (CBFs) [8], [9] provide hard safety assurances. Interpretability is crucial for understanding model functionality and failures. Methods to achieve interpretability include using intrinsically interpretable models like decision trees or techniques like model distillation [10]. Recent work by [11] uses symbolic AI for explanations, while [12] applied distillation to convert DRL into decision trees.

In contrast, our KAN-based actor maintains interpretability by design and learns to improve performance by load balancing flows across paths with varying network conditions.

This paper is organized as follows: Section I introduces the paper and reviews related work, Section II details our approach of KAN based DRL, Section III presents results, and Section IV concludes the paper.

## II. INTERPRETABLE REINFORCEMENT LEARNING

We propose replacing the traditional black-box actor in an actor-critic agent with a single-layer KAN, as shown in Fig. 1. This approach leverages the Kolmogorov-Arnold representation theorem, which states that any continuous function can be expressed using univariate splines. In our implementation, each state feature feeds into these splines, whose outputs are summed and passed through another function to produce an action. This decomposition allows for clear interpretability by tracing how each input contributes to the decision. Despite its simplicity, our 1-layer KAN actor matches the performance of deeper multilayer perceptrons (MLPs). The critic remains a standard MLP, estimating the state value  $V(s)$ , and both components are trained end-to-end using Proximal Policy Optimization (PPO) [13], [14].

<sup>1</sup><https://github.com/safeml/sdwan>

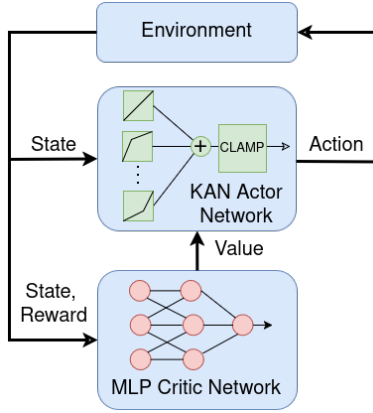


Fig. 1: KAN Actor and MLP Critic Architecture

**Environment:** The load balancing environment uses detailed packet-level simulations in NS3, connected to DRL scripts via ns3gym [15]. Every step, the DRL agent selects a target traffic ratio for the Internet or MPLS link. Observable parameters include: normalized total demand  $\lambda$ , demand change  $\Delta\lambda$ , and metrics for both links (MPLS and Internet), such as demand ( $\lambda_m, \lambda_i$ ), delay ( $d_m, d_i$ ), loss ( $l_m, l_i$ ), and utilization ( $u_m, u_i$ ). The number of flows on each link ( $n_m, n_i$ ) is also tracked, though background Internet traffic isn't directly observable—only its effects on delay and loss are measurable.

**State:** The State corresponds to the observed link conditions during the last step.  $\text{State} \in \{\lambda, \Delta\lambda, \lambda_m, 10d_m, 10l_m, u_m, \lambda_i, 10d_i, 10l_i, u_i\}$  (see Table I for definitions). Note that the delay and loss values are scaled by 10 to have values ranging from 0 to around 1 for DRL input.

**Action:** The action  $a$  is normalized ratio between -1 and 1, where values closer to 1 indicate proportionally more flows on MPLS, and closer to -1 indicate more on Internet:

$$a_{\text{current}} = 2 \frac{\frac{n_m}{C_m}}{\frac{n_m}{C_m} + \frac{n_i}{C_i}} - 1$$

Without normalization by capacity, the ratio could shift to extremes, hindering learning as some DRL libraries clip actions between  $\pm 1$ . We also use  $\text{clamp}(-1, 1)$  as the final activation. The DRL provides a target ratio and controller assigns flows to steer the current ratio toward the target, placing them on MPLS if below target, otherwise on Internet; ties are broken randomly.

#### A. Reward Design

We found reward design as tricky due to the variability in network metrics: reward volatility makes it difficult to assess learning progress. To address this, we normalize the reward using a scaling factor  $S = \frac{\beta}{\lambda^\alpha}$ , i.e., we normalize by average demand—observable unlike background load.

We primarily employ negative rewards to discourage detrimental actions leading to congestion. Two key approaches were considered: throughput utility maximization and loss minimization. Throughput alone is insufficient due to varying demand, leading us to develop a utility function capturing each flow's satisfaction as  $\max(1, \frac{b^k}{b_{\text{rate}}^k})$ , where  $b_{\text{rate}}^k$  and  $b^k$  are

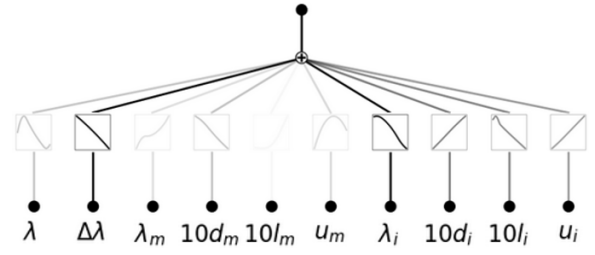


Fig. 2: PPO-KAN trained activations using loss reward

| Parameter                                             | Notation                         |
|-------------------------------------------------------|----------------------------------|
| Capacities MPLS, Internet, Total                      | $C_m, C_i, C$                    |
| Flow index                                            | $k$                              |
| Number of flows placed on MPLS, Internet              | $n_m, n_i$                       |
| Target normalised flow ratio                          | $a$                              |
| Average main demand load, background                  | $\hat{\lambda}, \hat{\lambda}_b$ |
| Demand load: overall, MPLS, Internet during last step | $\lambda, \lambda_m, \lambda_i$  |
| Delta load during last step                           | $\Delta\lambda$                  |
| Average delay: overall, MPLS, Internet                | $d, d_m, d_i$                    |
| Average loss: overall, MPLS, Internet                 | $l, l_m, l_i$                    |
| Link utilisation: MPLS, Internet                      | $u_m, u_i$                       |

TABLE I: Notation Table

desired and achieved bitrates of flow  $k$ . The reward is then calculated as:

$$-S \left( 1 - \frac{1}{n} \sum_k \max(1, \frac{b^k}{b_{\text{rate}}^k}) \right)$$

For loss minimization, we use  $-10 \cdot S \cdot l$ , which is simpler and effective, leveraging the fact that TCP-based traffic adjusts throughput based on loss signals.

#### B. Extracting Interpretable Policies

To extract interpretable policies, we propose 2 methods shown in Figure 3. We limit our symbolic space search to using monotonous functions:  $x, x^2, x^3, \text{sqrt}, \exp, \log, \tanh$ .

**KAN-symbolic:** This method trains a 1-layer KAN-based DRL actor using PPO (PPO-KAN). Fig. 2 shows an example of trained activation functions using packet loss as reward. The line intensity indicates the importance of the features. Thus, in case of 1-layer KAN, the model itself is highly interpretable. For more interpretability, the activation functions are extracted via regression and we call it KAN-symbolic. For throughput utility maximization, the policy is:  $a \approx$

$$1.3 - \frac{\lambda}{5} - \frac{(0.9\Delta\lambda + 1)^2}{3} - \frac{2\lambda_i}{5} + \frac{3(10d_i + 0.6)^2}{32} - \frac{30l_i}{32} \quad (1)$$

The policy prioritizes MPLS initially or during high delays, but shifts traffic to the Internet link as load increases. Loss impacts are minimal due to low magnitude of its term.

For loss minimization, the policy is:

$$a \approx 1.9 - 1.1 \left( \frac{2\Delta\lambda}{3} + 1 \right)^2 - \frac{2\lambda_i}{5} + \frac{10d_i}{3} + \frac{u_i}{10} \quad (2)$$

This policy favors MPLS initially but increases Internet usage with higher load and its delta. Increasing Internet delay and utilisation shifts some traffic to MPLS.

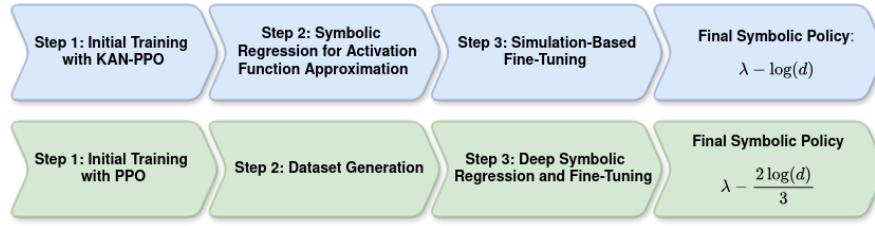


Fig. 3: Two methods for extracting symbolic policies

**Distillation of Symbolic Equations (PPO-DS):** This method uses PPO to train policies, then generates symbolic equations via auto-regressive models [16]. For throughput utility max., we have (action is clipped between  $[-1, 1]$ ):

$$a \approx \frac{2.0 - 8\Delta\lambda - 20d_m - 10l_i}{\exp(\lambda_i - u_i)} \quad (3)$$

This policy initially favors MPLS but shifts traffic to Internet as delta load and MPLS delay increase. When the gap between the placed load and link utilisation on Internet increases then  $a$  decreases, resulting in more load on Internet.

For loss minimization:

$$a \approx \exp\left(-\frac{(11\Delta\lambda + 10l_i)^2}{4}\right) \quad (4)$$

This policy prioritizes MPLS initially but increases Internet usage with higher delta load. Note  $a \geq 0$  always, i.e., MPLS has relatively more normalised proportion of traffic.

Counter-intuitively, some policies put more load on Internet when Internet loss increases. It is likely because after packet losses too many TCP flows backoff and sending new flows can capture the available capacity. Otherwise it will be taken by the background flows.

### III. EVALUATION

We evaluated our approach using detailed NS-3 packet-level simulations. The simulation environment and DRL training parameters are summarized in Table II. CleanRL library for DRL was used after modifications to implement a KAN-based actor instead of the default MLP-based one. Our traffic model employed TCP ON-OFF flows as non-homogeneous Poisson processes with time-varying arrival rates following a sine-wave pattern within episodes. The average rate for each episode  $\hat{\lambda}$  was randomly varied for both inter-episode and intra-episode variations to simulate dynamic load conditions. Depending on the episode's average  $\hat{\lambda}$ , traffic resulted in a sine-like pattern with randomly chosen phase, troughs between 7–12 Mbps, and peaks between 17–27 Mbps. Traffic demands resulted in approx. 0.3 to 1.3 times the combined network capacity, with background traffic generated via a Poisson process resulting in 5–8 Mbps. This setup captured realistic network behavior, where elastic traffic responded to congestion by experiencing delays and packet losses, prompting TCP flows to back off until sufficient bandwidth was available.

We compared reward signals of various policies after training and extracting their symbolic forms over 100 episodes

| Parameter                   | Value                                     |
|-----------------------------|-------------------------------------------|
| Capacity MPLS, Internet     | 6, 15 Mbps                                |
| Episode, step               | 2s steps, 50 steps per episode            |
| Main traffic                | wave of 7-17 Mbps to 12-27 Mbps           |
| Background traffic          | 5 to 8 Mbps, Poisson                      |
| Router Queue Size           | 100 packets                               |
| PPO Hyper-parameters        | [13], Actor, Critic, 2 layers of 64 units |
| Training steps              | 100000                                    |
| Learning rate               | 3e-3 with annealing                       |
| $\alpha, \beta, b_{rate}^k$ | 1.0, 0.2, 150 kbps                        |
| Steps per rollout           | 50                                        |
| gamma, gae lambda           | 0.99, 0.95                                |
| update epochs, clip coeff.  | 16, 0.2                                   |

TABLE II: Simulation and Training Parameters

| Method       | Interpret-score<br>(terms, nesting) | Thr-utility<br>Reward              | Loss<br>Reward                      |
|--------------|-------------------------------------|------------------------------------|-------------------------------------|
| PPO          | low                                 | -6.28 $\pm$ 1.79                   | <b>-13.13 <math>\pm</math> 3.49</b> |
| PPO-KAN      | high (visual)                       | <b>-6.27 <math>\pm</math> 1.66</b> | -13.28 $\pm$ 3.13                   |
| KAN-symbolic | high (6,1)                          | -6.313 $\pm$ 1.65                  | -13.56 $\pm$ 3.37                   |
| PPO-DS       | high (4,2)                          | -6.29 $\pm$ 1.80                   | -13.35 $\pm$ 3.28                   |

TABLE III: Evaluation of policies on 100 episodes. Reward results are presented as Mean  $\pm$  Standard Deviation.

(5,000 steps). Table III presents the mean  $\pm$  SD for throughput-utility and loss objectives, along with an interpretability score (number of terms and nesting in equation) assessing policy transparency. PPO-KAN, utilizing a single-layer KAN actor, performed similarly to regular PPO with less than 1% variation on both metrics. This demonstrates that KAN's functional decomposition is sufficiently expressive for two-path load balancing without performance compromise. Symbolic policies also achieved comparable rewards and offered direct inspection of decision logic through closed-form equations.

Fig. 4 and Fig. 5 show evaluation of network metrics using DRL for load balancing, comparing trained policies against Unequal-Cost Multi-Path (UCMP) [17] and a baseline Flow Count Balancing (FCB). UCMP distributes traffic per flow using a hash function on flow identifiers (e.g., 5-tuple). FCB places flow in proportion to MPLS and Internet link capacities, while counting active flows on each link to maintain this ratio. CCDFs for throughput utility, packet loss, and delay show that PPO and PPO-KAN outperformed baselines in most aspects. PPO with loss minimization reward performed better in 80-90% of cases but slightly worse elsewhere due to its focus on minimizing losses. Packet losses were reduced, with 20% more cases achieving similar or better throughput utility compared to baselines and an improvement from 22% to around 30% in cases when there was no-loss. Extracted symbolic policies (KAN-symbolic, PPO-DS) performed similarly to their parent policies, surpassing FCB and UCMP, demonstrating effective

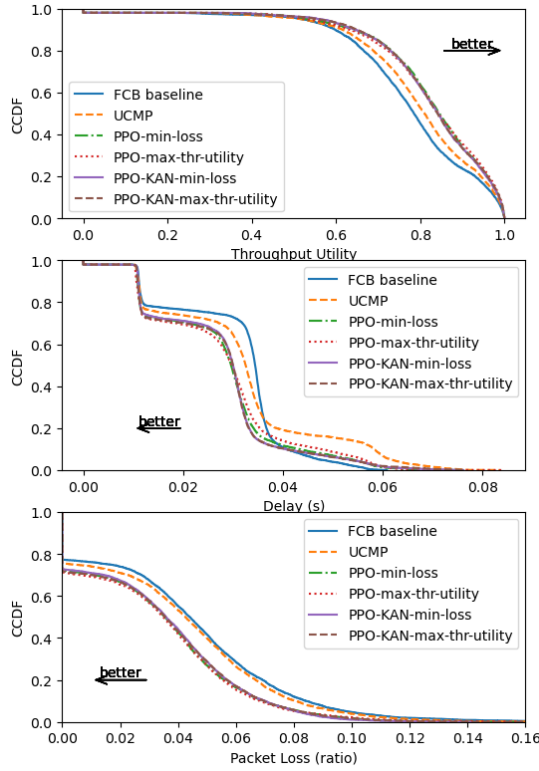


Fig. 4: CCDF of network metrics for PPO and PPO-KAN load balancing while maintaining interpretability. Thus, extracted policies behave very near to their PPO or PPO-KAN parents, from whom they were extracted, and they surpass the baselines.

#### IV. CONCLUSION

We proposed the use of KAN for interpretable RL in load balancing problems. We showed the advantage of combining KAN with PPO to learn policies that maximize throughput utility and minimize loss while being interpretable. We also showed that for load balancing, we could extract well performing symbolic policies. We believe that our approach can be applied to other networking problems. Future work includes evaluating our approach on more complex problems.

#### ACKNOWLEDGMENT

This work has been supported by grant ANR-21-CE25-0005 from the Agence Nationale de la Recherche, France for the SAFE project.

#### REFERENCES

- [1] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, and M. Tegmark, “KAN: Kolmogorov-arnold networks,” *arXiv preprint:2404.19756*, 2024.
- [2] S. Yasukawa, A. Farrel, and O. Komolafe, “An analysis of scaling issues in MPLS-TE core networks,” Tech. Rep., 2009.
- [3] D. Awduche, L. Berger, D. Gan, T. Li, V. Srinivasan, and G. Swallow, “RSVP-TE: extensions to RSVP for LSP tunnels,” Tech. Rep., 2001.
- [4] C. Filsfil, P. Camarillo, J. Leddy, D. Voyer, S. Matsushima, and Z. Li, “RFC 8986: Segment Routing over IPv6 (SRv6) Network Programming,” USA, 2021.
- [5] H. Wang, H. Xie, L. Qiu, Y. R. Yang, Y. Zhang, and A. Greenberg, “COPE: Traffic engineering in dynamic networks,” in *ACM SIGCOMM*, 2006, pp. 99–110.

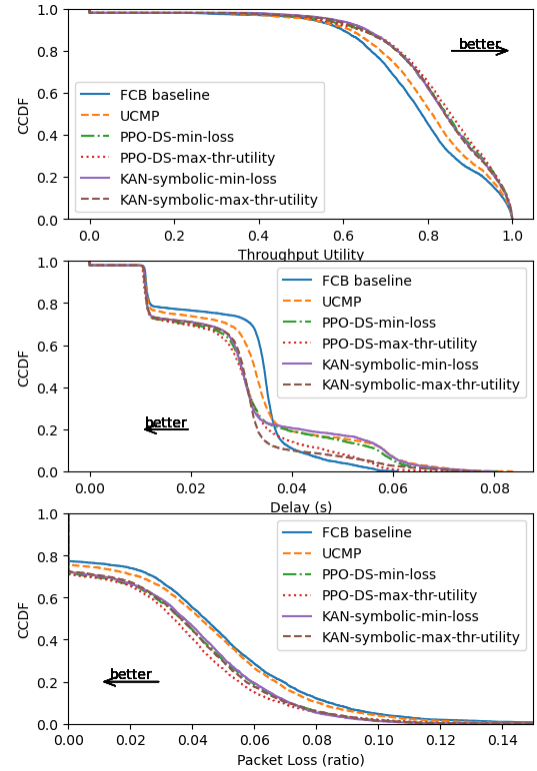


Fig. 5: CCDF of network metrics for extracted symbolic policies

- [6] G. Stampa, M. Arias, D. Sánchez-Charles, V. Muntés-Mulero, and A. Cabellos, “A deep-reinforcement learning approach for software-defined networking routing optimization,” *arXiv preprint:1709.07080*, 2017.
- [7] C. Tessler, D. J. Mankowitz, and S. Mannor, “Reward constrained policy optimization,” *arXiv preprint:1805.11074*, 2018.
- [8] L. Dinh, P. T. A. Quang, and J. Leguay, “Towards safe load balancing based on control barrier functions and deep reinforcement learning,” in *IEEE NOMS*, 2024, pp. 1–9.
- [9] —, “Safe load balancing in software-defined-networking,” *Computer Communications*, vol. 229, p. 107985, 2025.
- [10] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” *arXiv preprint:1503.02531*, 2015.
- [11] A. Duttgupta, M. Jabbari, C. Fiandrino, M. Fiore, J. Widmer *et al.*, “SYMBXRL: Symbolic Explainable Deep Reinforcement Learning for Mobile Networks,” in *IEEE INFOCOM*, 2025.
- [12] Z. Meng, M. Wang, J. Bai, M. Xu, H. Mao, and H. Hu, “Interpreting deep learning-based networking systems,” in *ACM SIGCOMM*, 2020, pp. 154–171.
- [13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint:1707.06347*, 2017.
- [14] V. A. Kich, J. A. Bottega, R. Steinmetz, R. B. Grando, A. Yorozu, and A. Ohya, “Kolmogorov-arnold network for online reinforcement learning,” *arXiv preprint arXiv:2408.04841*, 2024.
- [15] P. Gawtowicz and A. Zubow, “ns-3 meets OpenAI Gym: The Playground for Machine Learning in Networking Research,” in *ACM MSWiM*, November 2019.
- [16] B. K. Petersen, M. Landajuela, T. N. Mundhenk, C. P. Santiago, S. Kim, and J. T. Kim, “Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients,” in *ICLR*. OpenReview.net.
- [17] Huawei Technologies Co., Ltd., “UCMP Description – Unequal Cost Multi-Path (UCMP),” 2024, [Online]. Available: <https://support.huawei.com/enterprise/en/doc/EDOC1100367113/8bce14f6/allowbreakucmp-description> [Accessed: Sep. 1, 2025].